Beyond Imitation: Self-Improving Robot Policies via Off-Policy Q-Planning

Anonymous Author(s)
Affiliation
Address
email

Abstract: Behaviour Cloning (BC) has driven remarkable progress in robot manipulation, yet it is fundamentally limited by its inability to self-improve: a policy that fails cannot learn from that failure without additional human demonstrations. Reinforcement Learning offers a path to self-improvement but has proven difficult to scale to the multi-billion-parameter models underpinning modern robot policies. We propose Q-Planning, which equips a large visuomotor BC policy with a small off-policy Q-function. Q-learning is not restricted to successful demonstrations the way BC is, so the same Q-function can be trained on demos and later absorb both successful and failed deployment rollouts without ever imitating them. We exploit this asymmetry to enable value-guided action selection at inference and online self-improvement that fine-tunes only the Q-function, without ever updating the BC. Evaluated on LIBERO and RoboTwin, six iterations of online self-improvement progressively lift LIBERO-10 success from 93% to 99% while shortening successful episodes by 11%, updating only the Q-function; offline Q-Planning also improves over the BC policy on 4 of 5 benchmarks ($+1.4$pp on the LIBERO aggregate, $+4.8$pp on RoboTwin).

Keywords: Behaviour Cloning, Q-Learning, Model Predictive Control, Self-Improvement, Vision-Language-Action Models

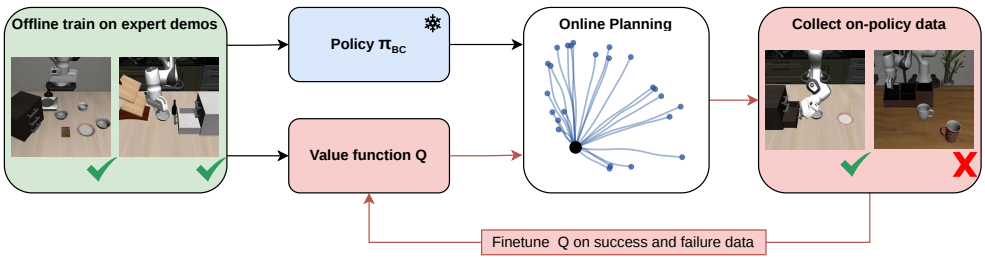

Figure 1: **Q-Planning overview.** *Left:* at inference, the frozen BC policy samples N candidate action chunks; the Q-function scores them and MPPI aggregates the chunk that is executed in the environment. *Right:* rollouts (successful and failed) are appended to a replay buffer and used to fine-tune *only* the Q-function. The updated Q_ϕ returns to the next inference iteration; the BC policy is never updated.

1 Introduction

Robot manipulation has progressed rapidly through large-scale Behaviour Cloning (BC), with vision-language-action (VLA) models such as RT-2 [1], Octo [2], $\pi^*_{0.6}$ [3], Diffusion Policy [4] and FastWAM [5] demonstrating impressive zero-shot generalisation on real-world tasks. This success, however, comes at a structural cost: BC policies are bounded by the data they were trained

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

on. A trained BC policy that fails on a task cannot learn from that failure without additional human teleoperation, and there is no mechanism by which rollouts collected during deployment improve subsequent behaviour. As we scale toward models with billions of parameters trained on millions of demonstrations, this *demo ceiling* becomes the dominant bottleneck for further progress.

The natural fix, Reinforcement Learning fine-tuning of the policy itself, has been pursued aggressively in the past year [6, 7, 8, 9, 10, 11, 12, 13, 14]. Yet updating a multi-billion-parameter VLA on sparse-reward, on-policy data is expensive, brittle, and can silently degrade the very BC prior that gave the policy its competence in the first place [15]. An alternative line of work, value-based planning [16, 17, 18, 19], sidesteps policy updates by re-ranking action candidates at inference time, but has not been demonstrated at VLA scale, where high-dimensional flow-matching action spaces and long horizons make planning over random proposals intractable.

Our insight is that the actor and the critic can be decoupled *and trained on different data*. A BC policy can only be trained on successful demonstrations (failure trajectories are not what we want to imitate), but an off-policy Q-function has no such restriction: it can be trained on any trajectory, successful or failed, because it estimates value rather than imitates actions. This asymmetry is what makes the self-improvement loop efficient: we can keep the expensive BC policy frozen and absorb all deployment signal, including failures, into a much smaller off-policy Q-function. Figure 1 illustrates this loop.

To combine the two components at inference we use MPPI [16], warm-started from BC samples (to localise the search where the BC's proposal distribution puts mass) and with temporal smoothing of injected noise (to keep candidate chunks inside the Q-function's training support). These three pieces, the off-policy Q-function, MPPI warm-started from BC, and temporal smoothing, together turn a frozen BC policy into a self-improving system. Concretely, on a hard 10-task LIBERO benchmark, self-improvement progressively lifts success from 93% to $\mathbf{99}\%$ in just 600 collected episodes (6 iterations of 100 episodes each), with the BC parameters frozen throughout. Offline (before any self-improvement), Q-Planning already improves over the BC policy on 4 of 5 benchmarks ($+1.4$pp on the LIBERO aggregate; $+4.8$pp on RoboTwin). Our contributions are:

1. **An off-policy Q-function for large BC policies.** A Q-chunking architecture with HL-Gauss categorical outputs and its own DinoV2 [20] and T5 [21] encoders, trained on the same demonstrations used to train the BC policy.
2. **Tractable Q-guided planning at VLA scale.** A temporal-smoothed MPPI planner with BC warm-starting that re-ranks flow-matching action chunks at inference, made efficient by re-using a single encoder forward pass for all N trajectory samples.
3. **Self-improvement without touching the BC.** A loop that turns 600 deployment episodes into Q-only updates, lifting LIBERO-10 success from 93% to 99% ($+9$pp over the frozen FastWAM policy).

2 Related work

Large-scale BC and VLA policies. The dominant paradigm for general-purpose robot manipulation is large-scale Behaviour Cloning on teleoperated demonstrations [22, 1, 2, 23, 4, 3, 5, 24]. These policies excel at imitating the demonstration distribution but inherit its ceiling: they cannot incorporate failure observations collected during deployment, and out-of-distribution states reliably degrade performance [15, 25]. Q-Planning treats such a BC policy as a frozen action proposal distribution and gives it a value function it never had.

Value functions and planning for control. Q-learning [26, 27, 28] and value-based planners such as MPPI [16], TD-MPC [17, 18] and MuZero [19] have demonstrated strong control performance in low-dimensional or simulated regimes, but extending them to VLA-scale manipulation has been hindered by the cost of online planning over high-dimensional action spaces. Recent work has explored pre-trained values as guidance for robotic foundation models [29, 30, 31, 32, 33], but does

not address how to use the value function to actively self-improve via online iteration. Q-Planning closes this gap.

Self-improvement and RL fine-tuning of robot policies. An active body of work fine-tunes large robot policies with online [6, 7, 8, 9, 11, 10, 12, 14, 34] or offline [35, 13, 36, 37] RL, and several recent systems iteratively co-improve a policy with a world model [38, 39, 40, 24]. All of these methods update the policy parameters, often the entire VLA, which is expensive at scale and risks degrading the BC prior. Q-Planning departs from this consensus: only the Q-function is updated during self-improvement, while the BC policy is left untouched.

3 Method

Overview. Q-Planning consists of three components: an off-policy Q-function over action chunks (Sec. 3.2), a temporal-smoothed MPPI planner with BC warm-starting (Sec. 3.3), and a self-improvement loop that fine-tunes only the Q-function (Sec. 3.4). At every stage the base BC policy is frozen.

3.1 Problem setting

We address language-conditioned manipulation in a partially-observed MDP with action $\mathbf{a}_t \in \mathbb{R}^{d_a}$, observation $\mathbf{o}_t$ (RGB images from one or more cameras), natural-language instruction $\boldsymbol{\ell}$, and per-step terminal indicator $d_t \in \{0, 1\}$ together with sparse terminal reward $r_t \in \{0, 1\}$ that is 1 only on the timestep at which the task is completed successfully. Following recent VLA work [4, 3, 5], the BC policy π^{BC} outputs an *action chunk* $\mathbf{a}_{t:t+H} \in \mathbb{R}^{H \times d_a}$ of length H at each planning step. We assume access to a dataset $\mathcal{D}_{\text{BC}}$ of successful demonstrations used to train π^{BC}. Our goal is to improve policy performance with minimal additional human supervision: we may collect rollouts with our own planner and update value-side parameters, but we never re-collect demonstrations and never update π^{BC}.

3.2 Off-policy Q-function over action chunks

Motivation. Standard scalar Q-regression under sparse terminal rewards is unstable [41]: most transitions carry no reward signal, and the long horizons of manipulation tasks amplify bootstrapping error. We therefore (i) treat the length-H action chunk as a single super-action so the effective bootstrapping horizon shrinks by a factor of H (Q-chunking [42]), and (ii) replace scalar regression with HL-Gauss categorical regression [41], which projects each scalar target onto a fixed grid of bins via a Gaussian kernel and trains the Q-network with a cross-entropy objective.

Module design. The Q-function has its own visual and language encoders, parameter-disjoint from the BC policy: $\mathbf{z}_t^{\text{vis}} = E_\phi^{\text{vis}}(\mathbf{o}_t)$ uses DinoV2 and $\mathbf{z}^{\text{txt}} = E_\phi^{\text{txt}}(\boldsymbol{\ell})$ uses T5. The Q-network $Q_\phi(\mathbf{o}_t, \boldsymbol{\ell}, \mathbf{a}_{t:t+H})$ is a transformer decoder that cross-attends to $[\mathbf{z}_t^{\text{vis}}; \mathbf{z}^{\text{txt}}]$ and takes the candidate action chunk as a sequence of query tokens. The decoder output is projected to B HL-Gauss logits ℓ_ϕ; the predicted scalar value is the expectation of the softmaxed categorical distribution over the bin grid $\{v_b\}_{b=1}^B$ of bin centres uniformly spaced in $[v_{\min}, v_{\max}]$:

$$Q_\phi(\mathbf{o}_t, \boldsymbol{\ell}, \mathbf{a}_{t:t+H}) = \sum_{b=1}^{B} v_b \cdot \text{softmax}\big(\ell_\phi(\mathbf{o}_t, \boldsymbol{\ell}, \mathbf{a}_{t:t+H})\big)_b. \tag{1}$$

Training. Following the Q-chunking formulation [42], we train Q_ϕ on chunked transitions $(\mathbf{o}_t, \mathbf{a}_{t:t+H}, r_{t:t+H}, \mathbf{o}_{t+H}, \mathbf{a}_{t+H:t+2H})$ sampled from a buffer $\mathcal{D}$, initialised to the demonstration dataset $\mathcal{D}_{\text{BC}}$ and later expanded with online rollouts (Sec. 3.4). At training time we do not query π^{BC} for a bootstrap action: we use the next chunk $\mathbf{a}_{t+H:t+2H}$ directly from the buffer, which when the buffer is $\mathcal{D}_{\text{BC}}$ matches what π^{BC} was trained to imitate and is therefore an unbiased sample of

π^{BC} in expectation. The chunked squared-error Bellman loss is

$$\mathcal{L}(\phi) = \mathbb{E}_{\mathcal{D}}\left[\left(Q_\phi(\mathbf{o}_t, \boldsymbol{\ell}, \mathbf{a}_{t:t+H}) - \sum_{t'=0}^{H-1} \gamma^{t'}\, r_{t+t'} - \gamma^H\, Q_{\bar{\phi}}(\mathbf{o}_{t+H}, \boldsymbol{\ell}, \mathbf{a}_{t+H:t+2H})\right)^2\right], \quad (2)$$

where $\bar{\phi}$ are exponential-moving-average target parameters. In practice we minimise the HL-Gauss cross-entropy projection of the right-hand-side target onto the bin grid rather than the squared error [41], which stabilises learning under sparse, bimodal returns. Critically, this loss applies unchanged when $\mathcal{D}$ later includes online rollouts containing failures (Sec. 3.4), because $Q^{\pi^{\text{BC}}}$ is well-defined for any state-action input regardless of where the data came from. This is the asymmetry that lets us train Q_ϕ on data π^{BC} itself could not be trained on.

Architectural decoupling. Keeping the Q-function's encoders parameter-disjoint from π^{BC} is what enables Q-Planning's self-improvement loop: failures in one network cannot corrupt the other, and the Q-function can be updated freely without ever touching the BC weights. Using large-scale pre-trained vision and language backbones (DinoV2 and T5) lets the Q-function inherit broadly-applicable inductive biases from web-scale supervision. An architecture diagram and training-stability arguments are provided in Appendix A.

3.3 Temporal-smoothed MPPI

Motivation. Given Q_ϕ, the natural inference-time strategy is to sample N action chunks per planning step, score them with Q_ϕ, and aggregate via MPPI weights. Two issues break the naive recipe at VLA scale: flow-matching BC heads produce near-identical samples (we mitigate this by running only 5 denoising steps, which also halves latency), and replacing BC samples with random noise yields chunks far outside Q_ϕ's training support. We therefore (i) *warm-start* the proposal from BC samples to localise the search on the manifold the BC already finds plausible (Sec. 4.3), and (ii) apply *temporal smoothing* to injected noise so perturbed chunks stay temporally coherent. Figure 2 illustrates the effect on sample diversity.

Module design. We run T iterations of MPPI [16] per planning step. At iteration i, the proposal distribution at planning step t has mean $\bar{\mathbf{a}}^{(i)}_{t:t+H} \in \mathbb{R}^{H\times d_a}$, initialised at iteration $i{=}0$ to the BC sample $\bar{\mathbf{a}}^{(0)}_{t:t+H} = \pi^{\text{BC}}(\mathbf{o}_t, \boldsymbol{\ell})$ (the warm-start), and is perturbed by additive temporally-smoothed Gaussian noise:

$$\begin{aligned} \mathbf{a}^{(n,i)}_{t:t+H} &= \bar{\mathbf{a}}^{(i)}_{t:t+H} + G_\sigma *_k \boldsymbol{\xi}^{(n)}, \\ \boldsymbol{\xi}^{(n)} &\in \mathbb{R}^{H\times d_a}, \;\; \boldsymbol{\xi}^{(n)}_{k,j} \sim \mathcal{N}(0,1), \;\; n = 1, \ldots, N. \end{aligned} \quad (3)$$

where G_σ is a 1-D Gaussian kernel of standard deviation σ convolved along the chunk axis k (so the per-dimension noise is correlated across the H planning steps). The MPPI weights are computed from Q-scores with temperature λ and used to update the proposal mean:

$$\begin{aligned} w^{(n,i)} &\propto \exp\!\big(Q_\phi(\mathbf{o}_t, \boldsymbol{\ell}, \mathbf{a}^{(n,i)}_{t:t+H})/\lambda\big), \\ \bar{\mathbf{a}}^{(i+1)}_{t:t+H} &= \sum_{n=1}^{N} w^{(n,i)}\, \mathbf{a}^{(n,i)}_{t:t+H}. \end{aligned} \quad (4)$$

After T iterations, the converged mean $\bar{\mathbf{a}}^{(T)}_{t:t+H}$ is executed in the environment up to a re-planning interval. The Q-network is evaluated for all N trajectory samples in a single forward pass: vision and text tokens are

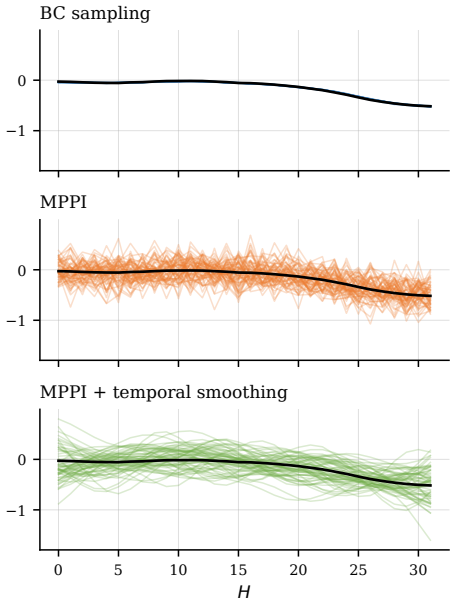

Figure 2: **Action-chunk diversity** on one LIBERO-10 planning step (z-translation; 64 candidates each). BC samples cluster tightly; vanilla MPPI spreads but is jagged; temporal smoothing keeps spread while staying coherent.

encoded once per iteration and the N samples enter the decoder as N parallel query sequences, so the per-iteration planning cost is dominated by one encoder pass plus one batched decoder pass.

3.4 Self-improvement loop

Motivation. A BC policy trained only on successful demonstrations has never seen the failure modes that emerge under autonomous execution. Q-Planning closes this loop by deploying the planner, collecting both successful *and* failed rollouts, and using them to refine *only* the Q-function. This keeps the cost of one self-improvement iteration proportional to the size of the Q-function ($\sim$ 1B parameters) rather than the BC policy (multi-billion; App. B), which is the expensive component to train.

Algorithm. Algorithm 1 summarises the loop. Each iteration runs the planner for M episodes per task on the target benchmark, appends the collected transitions (regardless of success) to a replay buffer $\mathcal{D}$, and refines Q_ϕ for S gradient steps on $\mathcal{D}$ using the loss in Eq. 2.

Algorithm 1: Q-Planning Self-Improvement

$\mathcal{D} \leftarrow \mathcal{D}_{\text{BC}}$;
for *iteration* $i = 1, 2, \ldots$ **do**
  `// (1) Collect rollouts under Q-Planning;`
  for *task* $k \in \mathcal{T}$, *episode* $j = 1 \ldots M$ **do**
    Reset env; receive $\mathbf{o}_0$;
    while *episode not done* **do**
      Plan iteratively with MPPI: $\bar{\mathbf{a}}^{(T)}_{t:t+H}$ via Eqs. 3, 4;
      Execute $\bar{\mathbf{a}}^{(T)}_{t:t+H}$; observe $r_{t:t+H}, \mathbf{o}_{t+H}$;
    end
    Append the full episode to $\mathcal{D}$;
  end
  `// (2) Refine Q only (BC frozen);`
  for $s = 1 \ldots S$ **do**
    Update ϕ on a minibatch from $\mathcal{D}$ via $\mathcal{L}(\phi)$ (Eq. 2);
    EMA update $\bar{\phi} \leftarrow \eta\,\phi + (1{-}\eta)\,\bar{\phi}$;
  end
end

Decoupling the policy from the value function lets the Q-function absorb failure signal that is not present in $\mathcal{D}_{\text{BC}}$, while leaving the BC's behavioural prior intact. The loop can be folded into deployment by alternating data collection with cheap Q-only updates, since the BC policy requires no retraining. We use $M = 100$ episodes per task per iteration, $S = 200$ Q-only gradient steps per iteration, and EMA target rate $\eta = 0.005$; full hyperparameters are in Appendix B.

4 Experimental results

Our experiments target the three core questions of an evidence-driven evaluation: **(Q1)** Does Q-Planning preserve the BC policy's offline performance while opening a path to online self-improvement? **(Q2)** Which design choices (temporal smoothing, BC warm-starting) drive the result? **(Q3)** Does Q-Planning enable self-improvement that the frozen BC cannot achieve on its own?

4.1 Experimental setup

We evaluate on two simulated manipulation benchmarks: the four LIBERO [43] suites (Spatial, Object, Goal, and the 10 hard long-horizon tasks of LIBERO-10), and 47 RoboTwin [44] tasks (a subset of the full RoboTwin suite that we have working evaluations for). All reported success rates and episode lengths aggregate over 20 episodes per task. For the BC policy we use the publicly-released FastWAM [5] weights, frozen throughout. The Q-function is trained on the same success-

ful demonstrations that FastWAM was trained on (no additional data), for 12k gradient steps on LIBERO and 20k on RoboTwin, on $4\times$ H200 GPUs, until per-task Q-values converge. LIBERO is evaluated with the standard episode budget of 540 environment steps (rather than the 700 steps used in the FastWAM paper); absolute LIBERO success rates therefore sit a few percentage points below FastWAM's reported numbers, and this protocol applies identically to both baseline and Q-Planning rows.

4.2 Which action source for planning?

We first ask which action source the Q-function should score: raw BC samples, vanilla MPPI noise, or temporally smoothed MPPI noise (the three regimes visualised in Figure 2). We run all three variants on LIBERO-10 with $N{=}5$ BC samples and $N{=}64$ MPPI candidates, and report the unguided FastWAM policy as a reference. Table 1 reports the result.

Table 1: **Action source comparison on LIBERO-10** (10 tasks, 20 episodes/task). The reference row is the frozen FastWAM with no value guidance. Q-Planning variants below differ only in how trajectory samples are drawn. MPPI with temporal smoothing exceeds the baseline by $+3$pp; raw BC sampling and vanilla MPPI do not.

Action source	Success ↑	Mean ep. length ↓
FastWAM (reference, no Q)	90.0	274
BC sampling	91.5	276
MPPI	89.5	270
MPPI + temporal smoothing	**93.0**	261

Only MPPI with temporal smoothing meaningfully exceeds the BC policy offline ($+3$pp over FastWAM). Vanilla MPPI sits slightly below the baseline because jagged candidates fall outside the Q-function's training support, making Q-scores unreliable; temporal smoothing restores in-distribution proposals, which both lifts the offline number and (Sec. 4.5) makes the self-improvement loop work.

4.3 Warm-starting is essential for tractable planning

We next isolate the contribution of BC warm-starting by replacing the iteration-0 proposal mean $\bar{\mathbf{a}}^{(0)}_{t:t+H} = \pi^{\mathrm{BC}}(\mathbf{o}_t, \boldsymbol{\ell})$ with a zero-mean Gaussian and otherwise running the identical planner on LIBERO-10. Table 2 reports the result.

Table 2: **Warm-starting ablation on LIBERO-10.** Removing BC warm-starting collapses success from 93.0% to 3.3%: greedy MPPI search over uninformative random actions is intractable at the dimensionality and horizon of long-horizon manipulation. The handful of completed no-warm-start episodes succeed only on the simplest tasks.

Initialisation	Success ↑
MPPI w/o warm-start	3.3
MPPI + BC warm-start (ours)	**93.0**

The $+89.7$ percentage-point gap addresses Q2: in the action dimensions and horizons that BC policies routinely handle, a value-guided planner without a strong proposal distribution has effectively no chance of finding a successful trajectory by random search. Q-Planning's viability rests on the fact that the BC policy already proposes plausible candidates.

4.4 Main results: LIBERO and RoboTwin

Table 3 reports offline (no self-improvement) results across the four LIBERO suites and RoboTwin. Q-Planning improves over FastWAM on every benchmark we tested except the already-saturated LIBERO-Object (99.5 vs 100.0): $+1.4$pp on the LIBERO aggregate (winning 3 of 4 suites; $+3$pp

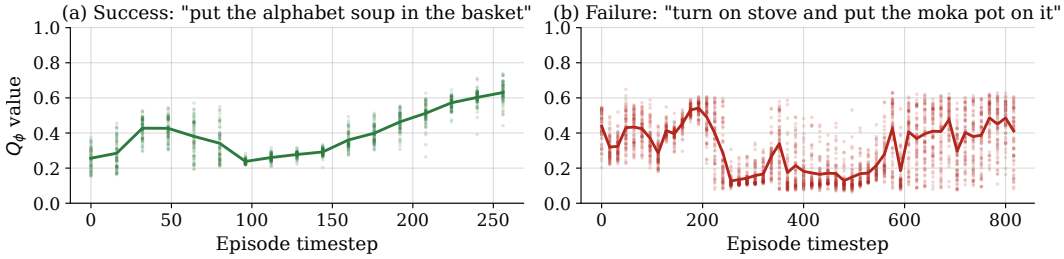

Figure 3: **Q-value over time** on two representative LIBERO-10 episodes. Solid lines show Q_ϕ of the executed chunk; faint scatter shows the N=64 MPPI candidates per planning step. On success (left), Q_ϕ climbs from ~ 0.26 to ~ 0.63; on failure (right) it oscillates in a lower range and never converges.

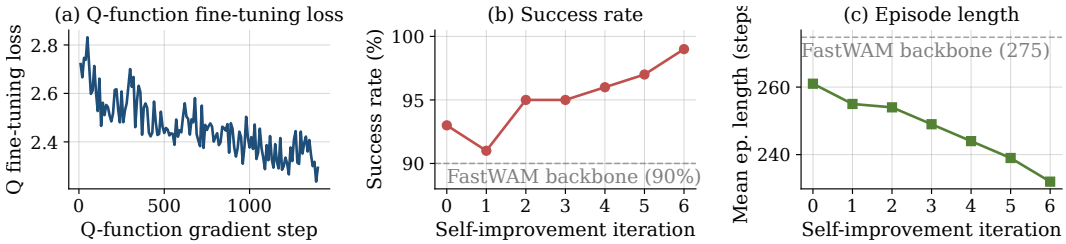

Figure 4: **Q-Planning self-improvement on LIBERO-10.** (a) Q-function fine-tuning loss across all gradient steps. (b) Policy success rate after each self-improvement iteration. (c) Mean successful episode length after each iteration. Iteration 0 is the pre-self-improvement baseline (matches the offline Q-Planning row of Table 3); iterations $1-6$ are after each successive round of 100 rollouts followed by 200 Q-only gradient steps. Across the six rounds, success rises from 93% to **99**% and mean successful episode length drops from 261 to **232** environment steps. Dashed lines mark the frozen FastWAM. The BC parameters are never updated.

on the hardest, LIBERO-10) and +4.8pp on RoboTwin. This answers Q1 cleanly: value-guided action selection strictly improves the BC policy on average across both simulation suites without ever updating its parameters. Figure 3 additionally visualises Q_ϕ on a successful and a failed rollout.

Table 3: **Offline main results** on the four LIBERO suites and RoboTwin. Per-row values are means of per-task success rates and per-task mean successful episode length. **Bold** marks the higher success rate. Q-Planning (offline) improves over the BC policy on 4 of 5 benchmarks, by +1.4pp on the LIBERO aggregate and +4.8pp on RoboTwin, without ever updating the BC.

	FastWAM [5]		Q-Planning (offline)	
Benchmark	Success ↑	Ep. len. ↓	Success ↑	Ep. len. ↓
LIBERO-Spatial	90.5	106	**91.5**	115
LIBERO-Object	**100.0**	138	99.5	139
LIBERO-Goal	97.0	107	**99.0**	110
LIBERO-10	90.0	274	**93.0**	261
RoboTwin (47 tasks)	83.2	220	**88.0**	231
Mean	92.1	–	**94.2**	–

4.5 Online self-improvement on LIBERO-10

We finally answer Q3 on LIBERO-10, the most demanding LIBERO suite. Starting from the same FastWAM and from a Q-function pre-trained on successful demonstrations only (iteration 0, matching the offline Q-Planning row of Table 3), we run the loop in Algorithm 1 with $M = 100$ episodes per iteration and $S = 200$ Q-updates per iteration. The BC policy is frozen throughout. Figure 4 shows the resulting curves.

Success rises from 93% at iteration 0 to 99% at iteration 6, while mean successful episode length shortens from 261 to 232 environment steps; the Q-function fine-tuning loss converges over the same window. The improvement is achieved without updating any BC weights: the only parameters that change between iteration 0 and iteration 6 are those of Q_ϕ.

5 Conclusion

We introduced Q-Planning, a route to self-improving robot manipulation policies that decouples the actor from the critic by pairing a frozen BC policy with a off-policy Q-function evaluated at inference via temporal-smoothed MPPI warm-started from BC samples. Offline, Q-Planning improves over the BC policy on 4 of 5 benchmarks ($+1.4$pp on the LIBERO aggregate, $+4.8$pp on RoboTwin); six iterations of online self-improvement that touch *only* the Q-function then progressively lift LIBERO-10 success from 93% to 99% ($+9$pp over the frozen FastWAM) while shortening successful episodes by 11%, evidence that a frozen BC policy can be improved through Q-function updates alone.

The asymmetry that makes this possible, namely that BC must train on successful demonstrations but an off-policy Q-function can train on any rollout, is what lets the self-improvement loop scale with Q_ϕ's size rather than the policy's. As VLA backbones grow toward 10B$+$ parameters, full-policy RL fine-tuning becomes prohibitive and risks degrading the BC prior; Q-Planning freezes the BC entirely and routes all deployment signal into a much smaller off-policy Q-function instead. Most immediately, a real-robot loop with a learned per-episode success classifier would test whether the simulation conclusions transfer; replicating self-improvement on the remaining LIBERO suites and RoboTwin, swapping the BC policy (Octo [2], $\pi^*_{0.6}$ [3], Diffusion Policy [4]), and a head-to-head against full-policy RL fine-tuning [9, 6] would each further validate the approach. We hope Q-Planning paves a way to self-improve large robot policies beyond the ceiling that imitation learning alone can reach.

6 Limitations

BC policy dependence. Q-Planning cannot bootstrap from scratch: removing the BC warm-start collapses success from 93% to 3.3% (Sec. 4.3). The method inherits the BC's action-distribution biases, and its gains scale with BC quality. The sparse terminal reward also assumes a reliable per-episode success detector, trivial in simulation but non-trivial on real hardware.

Online planning overhead. Q-Planning replaces single-sample BC inference with $NT{=}192$ candidate evaluations per planning step, adding decoder latency relative to direct BC inference (encoders are amortised). Two directions could remove this: (i) fold the Q-signal into the BC via policy-gradient fine-tuning, trading inference cost for training cost; (ii) replace MPPI with diverse direct BC sampling, which needs a policy that produces diverse action chunks (not diffusion or flow-matching; Sec. 4.2).

References

[1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In J. Tan, M. Toussaint, and K. Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

[2] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.

[3] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, et al. $\pi^*_{0.6}$: a VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[4] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023.

[5] T. Yuan, Z. Dong, Y. Liu, and H. Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.

[6] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, Q. Zhang, Z. Yu, G. Fan, et al. πrl: Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025.

[7] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, et al. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025.

[8] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.

[9] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025.

[10] Y. Li, X. Ma, J. Xu, Y. Cui, Z. Cui, Z. Han, L. Huang, T. Kong, Y. Liu, H. Niu, et al. Gr-rl: Going dexterous and precise for long-horizon robotic manipulation. *arXiv preprint arXiv:2512.01801*, 2025.

[11] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen. Improving vision-language-action model with online reinforcement learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15665–15672. IEEE, 2025.

[12] S. Tan, K. Dou, Y. Zhao, and P. Krähenbühl. Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*, 2025.

[13] M. S. Mark, T. Gao, G. G. Sampaio, M. K. Srirama, A. Sharma, C. Finn, and A. Kumar. Policy agnostic rl: Offline rl and online rl fine-tuning of any class and backbone. *arXiv preprint arXiv:2412.06685*, 2024.

[14] J. Liu, F. Gao, B. Wei, X. Chen, Q. Liao, Y. Wu, C. Yu, and Y. Wang. What can rl bring to vla generalization? an empirical study. *Advances in Neural Information Processing Systems*, 38: 97121–97151, 2026.

[15] A. Kumar, J. Hong, A. Singh, and S. Levine. Should i run offline reinforcement learning or behavioral cloning? In *International conference on learning representations*, 2021.

[16] G. Williams, A. Aldrich, and E. A. Theodorou. Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017.

[17] N. Hansen, X. Wang, and H. Su. Temporal difference learning for model predictive control. 2022.

[18] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control. 2024.

[19] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

[20] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.

[21] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[22] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[23] A. Padalkar, A. Pooley, A. Jain, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Singh, A. Brohan, et al. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv preprint arXiv:2310.08864*, 2023.

[24] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, A. Raju, et al. Robocat: A self-improving foundation agent for robotic manipulation. *arXiv preprint arXiv:2306.11706*, 2023.

[25] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg. Dart: Noise injection for robust imitation learning. In *Conference on robot learning*, pages 143–156. PMLR, 2017.

[26] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[27] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

[28] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[29] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.

[30] C. Bhateja, D. Guo, D. Ghosh, A. Singh, M. Tomar, Q. Vuong, Y. Chebotar, S. Levine, and A. Kumar. Robotic offline rl from internet videos via value-function pre-training. *arXiv preprint arXiv:2309.13041*, 2023.

[31] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang. Vip: Towards universal visual reward and representation via value-implicit pre-training. *arXiv preprint arXiv:2210.00030*, 2022.

[32] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023.

[33] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.

[34] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, et al. Self-improving vision-language-action models with data generation via residual rl. *arXiv preprint arXiv:2511.00091*, 2025.

[35] A. Kumar, A. Singh, F. Ebert, M. Nakamoto, Y. Yang, C. Finn, and S. Levine. Pre-training for robots: Offline rl enables learning new tasks from a handful of trials. *arXiv preprint arXiv:2210.05178*, 2022.

[36] J. Yang, M. S. Mark, B. Vu, A. Sharma, J. Bohg, and C. Finn. Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4804–4811. IEEE, 2024.

[37] Z. Hu, R. Wu, N. Enock, J. Li, R. Kadakia, Z. Erickson, and A. Kumar. Rac: Robot learning for long-horizon tasks by scaling recovery and correction. *arXiv preprint arXiv:2509.07953*, 2025.

[38] Y. Guo, T. Lee, L. X. Shi, J. Chen, P. Liang, and C. Finn. Vlaw: Iterative co-improvement of vision-language-action policy and world model. *arXiv preprint arXiv:2602.12063*, 2026.

[39] C. Liu, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen. Self-correcting vla: Online action refinement via sparse world imagination. *arXiv preprint arXiv:2602.21633*, 2026.

[40] E. Y.-T. Jia, W. Yuan, T. Shi, V. Guizilini, J. Mao, and Y. Wang. Dreamplan: Efficient reinforcement fine-tuning of vision-language planners via video world models. *arXiv preprint arXiv:2603.16860*, 2026.

[41] J. Farebrother, J. Orbay, Q. Vuong, A. A. Taïga, Y. Chebotar, T. Xiao, A. Irpan, S. Levine, P. S. Castro, A. Faust, et al. Stop regressing: Training value functions via classification for scalable deep rl. *arXiv preprint arXiv:2403.03950*, 2024.

[42] Q. Li, Z. P. Zhou, and S. Levine. Reinforcement learning with action chunking. *Advances in Neural Information Processing Systems*, 38:55518–55553, 2026.

[43] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[44] Y. Mu, T. Chen, Z. Chen, S. Peng, Z. Lan, Z. Gao, Z. Liang, Q. Yu, Y. Zou, M. Xu, et al. Robotwin: Dual-arm robot benchmark with generative digital twins. In *Proceedings of the computer vision and pattern recognition conference*, pages 27649–27660, 2025.

A Q-function architecture

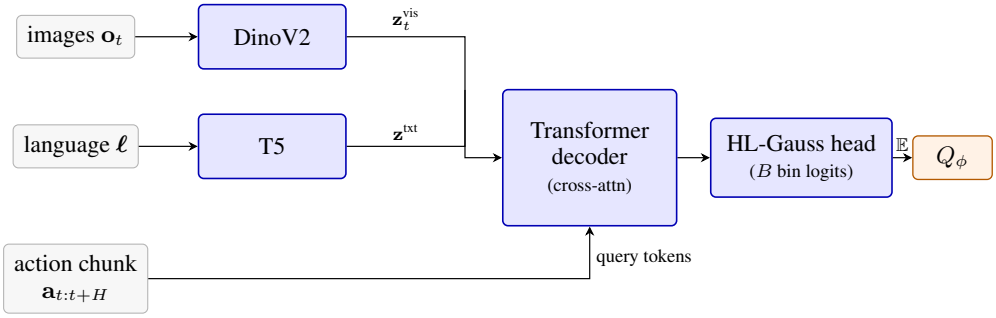

Figure 5: **Q-function architecture.** The Q-function Q_ϕ has its own DinoV2 visual encoder and T5 language encoder (parameter-disjoint from the BC policy). The transformer decoder cross-attends to visual tokens $\mathbf{z}_t^{\text{vis}}$ and language tokens $\mathbf{z}^{\text{txt}}$ (keys/values) and takes the candidate action chunk $\mathbf{a}_{t:t+H}$ as a sequence of query tokens. The HL-Gauss head outputs B bin logits over discounted returns and the scalar Q_ϕ is the expectation of the softmaxed bin distribution.

B Implementation details

Hyperparameters. We use $H = 16$ action-chunk length, $N = 64$ trajectory samples per planning step, $T = 3$ MPPI iterations, smoothing $\sigma = 2$, MPPI temperature $\lambda = 1$, discount $\gamma = 0.99$, and replan every H environment steps. When sampling action chunks from FastWAM during Q-Planning, we use 5 flow-matching denoising steps (rather than the default), which roughly halves BC inference latency and intentionally produces noisier samples that benefit the MPPI proposal distribution. The Q-function output uses $B = 101$ HL-Gauss bins on $[v_{\min}, v_{\max}] = [0, 1]$ with kernel width matched to bin spacing. Optimisation uses AdamW with learning rate $3{\times}10^{-4}$ and an EMA target rate of $\eta = 0.005$. Q-training runs on $4\times$ H200 GPUs for 12k gradient steps on LIBERO and 30k on RoboTwin before evaluation. One self-improvement iteration consists of $M = 100$ collected episodes followed by $S = 200$ Q-only gradient steps on the combined offline + online buffer; the BC policy is frozen for all S steps and across all iterations. LIBERO episodes are capped at 540 environment steps; RoboTwin episodes use the benchmark default.

Model size. The Q-function totals approximately 1B parameters (DinoV2 visual encoder $\sim$ 300M, T5 language encoder $\sim$ 220M, transformer decoder $\sim$ 500M). This is small relative to the multi-billion-parameter FastWAM policy, and accounts for why one self-improvement iteration (S=200 Q-only gradient steps) is dramatically cheaper than a full-policy gradient step.

Training stability. Two ingredients keep Q-training stable under sparse terminal rewards: HL-Gauss categorical regression removes the scale sensitivity of scalar MSE regression to sparse, bimodal returns, and Q-chunking shrinks the effective bootstrapping horizon by a factor of H, mitigating long-horizon variance blow-up.